ETH zürich

ETH Library

The Risk and Reward of Cross-Engine Query Partitioning: A Case for Partition-Aware Benchmarking

Conference Paper

Author(s):
Giurgiu, Ioana; Barrada, Soufiane; Gupta, Shaswat; Nidd, Michael E.

Publication date:
2026-05

Permanent link:
https://doi.org/https://doi.org/10.3929/ethz-c-000803213

Rights / license:
Creative Commons Attribution 4.0 International

Originally published in:
https://doi.org/10.1145/3810991.3811629

This page was generated automatically upon download from the ETH Zurich Research Collection.
For more information, please consult the Terms of use.

The Risk and Reward of Cross-Engine Query Partitioning: A Case for Partition-Aware Benchmarking

Ioana Giurgiu[1], Soufiane Barrada[2], Shaswat Gupta[1], Michael E. Nidd[1]
[1]IBM Research Europe, Zürich, Switzerland [2]ETH Zürich, Switzerland

Abstract

Query engines, whether deployed independently or behind a unified lakehouse interface, evaluate each query on a single engine and report the resulting latency. Whole-query routing treats queries as indivisible units of work, an assumption we challenge both empirically and methodologically. We conduct the first systematic upper-bound study of cross-engine query partitioning, exhaustively evaluating all valid two-fragment partitions across DuckDB and DataFusion on 751 analytical queries. We find that partitioning improves 40% of queries, with speedups up to 14.87× and workload-level gains of 25–33% when compared to the fastest engine. However, the partition space is sharply asymmetric: poor cuts can be orders of magnitude slower, revealing a pronounced risk–reward trade-off.

These findings demonstrate that whole-query evaluation systematically underestimates both the performance potential and the risk profile of heterogeneous systems. We propose a partition-aware benchmark protocol that reports three dimensions current evaluations miss: the performance ceiling, the cost of approaching it, and the risk of partition mis-selection.

CCS Concepts

• **Information systems → Database performance evaluation**.

Keywords

Cross-engine query partitioning, Partition-aware benchmarking

ACM Reference Format:
Ioana Giurgiu[1], Soufiane Barrada[2], Shaswat Gupta[1], Michael E. Nidd[1], [1]IBM Research Europe, Zürich, Switzerland [2]ETH Zürich, Switzerland . 2026. The Risk and Reward of Cross-Engine Query Partitioning: A Case for Partition-Aware Benchmarking. In *Testing Database Systems (DBTest '26), May 31-June 05, 2026, Bengaluru, India.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3810991.3811629

1 Introduction

Modern data platforms increasingly expose multiple heterogeneous query engines within a single lakehouse or polystore architecture. Columnar analytical engines, embedded runtimes, distributed processing frameworks, and specialized accelerators coexist behind unified query interfaces [10, 25] . The prevailing evaluation model in such environments remains simple: given a query, identify the single engine that executes it fastest and report that latency.

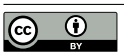
This whole-query routing assumption implicitly treats heterogeneous systems as collections of independent engines competing for ownership of a query. In practice, analytical queries are structurally heterogeneous: different fragments often exhibit distinct computational profiles. A scan–join-heavy prefix may favor one engine, while a lightweight aggregation or projection suffix may favor another. In this sense, heterogeneity operates at the fragment level rather than the query level. If the gains from fragment specialization exceed the cost of transferring intermediate results, partitioned execution should outperform any single-engine plan.

Despite this intuition, there has been little systematic empirical evaluation of the performance ceiling introduced by cross-engine partitioning. Existing systems employ heuristic or cost-based rules to split queries [3, 16, 18], but the magnitude of the attainable benefit and the associated risk remains unclear. As heterogeneous environments grow in complexity, understanding this ceiling becomes increasingly important for both optimization and benchmarking.

In this paper, we conduct the first exhaustive exploration of two-fragment query partitioning across DuckDB [22] and DataFusion [2]. For each query in three workloads (TPC-H, JOB, and StackOverflow), we enumerate all valid plan cut points and evaluate all possible engine assignments. Across 751 queries and 15,921 partition configurations, we compare the best partitioned execution against the best single-engine execution. By exhaustively enumerating all valid two-fragment cuts, our study quantifies the empirical upper bound of cross-engine partitioning under controlled assumptions.

Our results are striking. Partitioning reveals substantial and previously unmeasured headroom, as it improves 40% of queries, with speedups up to 14.87×. At the workload level, best cross-engine partitioning reduces total execution time by 25–33% compared to the fastest single engine. Based on these findings, our primary goal is not to characterize a specific engine pair, but to demonstrate that whole-query evaluation systematically underestimates both the performance potential and the risk profile of heterogeneous systems, and that partition-aware metrics are needed to surface these effects.

We make three contributions:
1. Partition-aware benchmarking perspective. We argue that whole-query routing systematically underestimates the performance potential of heterogeneous systems by ignoring intra-query specialization opportunities, and propose a partition-aware evaluation perspective that exposes these effects.
2. Empirical ceiling study. We conduct the first exhaustive exploration of two-fragment cross-engine partitions across two engines, quantifying the attainable performance ceiling and revealing a pronounced risk–reward asymmetry in the partition space.
3. Benchmark protocol. We propose a practical evaluation protocol for heterogeneous systems that reports three dimensions absent



This work is licensed under a Creative Commons Attribution 4.0 International License.
DBTest '26, Bengaluru, India
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2701-6/2026/05
https://doi.org/10.1145/3810991.3811629

from conventional benchmarks: the partition ceiling, the risk of partition mis-selection, and the cost of exploring the partition space.

2 Problem Setup

We consider a set of query engines $\mathcal{E} = \{E_1, \ldots, E_n\}$, each capable of executing SQL over a shared dataset but differing in architecture and optimizer strategy. The baseline *whole-query* approach executes a query Q on a single engine; its best latency is $\ell^*_{\text{single}}(Q) = \min_{E_i \in \mathcal{E}} \ell(Q, E_i)$.

A *two-fragment partition* splits the operator tree $T(Q)$ at a single cut edge into a lower fragment F_1 (closer to the base tables) and an upper fragment F_2, with an intermediate result R flowing across the cut. Each fragment is assigned to an engine; when assignments differ, R must be transferred. The total partitioned latency is:

$$\ell_{\text{part}} = \ell(F_1, \alpha(F_1)) + \ell(F_2, \alpha(F_2)) + \tau(R, \alpha(F_1) \to \alpha(F_2)),$$

where τ denotes the transfer cost and α represents the engine assignment. In our study, F_1 and F_2 are always assigned to different engines, so τ reflects the cost of serializing and transferring R (via Apache Arrow).

Our goal is to determine whether any two-fragment partition and engine assignment improves over single-engine execution. Let $\ell^*_{\text{part}}(Q) = \min_{(F_1,F_2) \in \text{cuts}(Q),\, \alpha} \ell_{\text{part}}(F_1, F_2, \alpha)$ denote the minimum partitioned latency over all valid cuts and cross-engine assignments. Partitioning is beneficial when $\ell^*_{\text{part}}(Q) < \ell^*_{\text{single}}(Q)$.

3 Methodology

Workload. We evaluate on three analytical workloads: TPC-H SF10 (247 queries), JOB (229 queries), and StackOverflow SF10 (SO, 275 queries). These queries have been selected from the SQLStorm [23] dataset and span high and medium complexities and diverse data characteristics.

Engines. Our environment consists of two query engines representing distinct architectural philosophies: (1) **DuckDB** [22]: a C++-based embedded columnar engine with vectorized execution and a cost-based optimizer, targeting analytical workloads; (2) **DataFusion** [2]: a Rust-based embedded columnar engine from the Apache Arrow ecosystem with Arrow-native data representation and a heuristic-based optimizer, also targeting analytical workloads.

All experiments run on a single node equipped with a 64-core AMD EPYC 7763 processor and 2 TB of main memory. We use DuckDB 1.2.1, Apache DataFusion 50.1.0, as well as Apache Arrow [1] for transfers between the engines.

Both engines operate on a shared dataset on identical hardware, isolating performance differences to engine execution models. DuckDB is substantially faster than DataFusion in single-engine execution for the majority of queries (87% on average), making the partitioning results more striking: the gains come not from balancing evenly matched engines, but from exploiting per-fragment advantages even when one engine dominates overall.

Partition enumeration. For each query, we obtain the logical query plan and identify all edges in the operator tree as candidate cut points. We enumerate all such cuts. For each partition, we consider DuckDB→DataFusion and DataFusion→DuckDB engine assignments. Across the full workload, this produces 15,921 configurations. On average, each query yields approximately 10.6 candidate cuts. We record the intermediate result size $|R_1|$ for each cut to analyze the relationship between transfer cost and partitioning benefit. We report complete results based on both engine assignments; the supplementary repository[1] additionally highlights the DuckDB→DataFusion direction, where partitioning yields the most pronounced improvements.

Measurement protocol. Each configuration is executed 5 times; we report median wall-clock latency, measuring F_1 execution, transfer, and F_2 execution separately.

Table 1: Total workload time under best single-engine routing vs. best partitioning.

Workload	Single-engine (s)	Oracle-best (s)	Ceiling gap
TPC-H	899.5	658.8	26.8%
JOB	488.0	362.2	25.8%
SO	2259.4	1514.2	33.0%

4 Results

We now analyze the outcomes of exhaustive two-fragment partitioning across the three workloads. Our analysis addresses four questions: (1) how often partitioning improves over the best single-engine execution and by how much; (2) how risky the partition space is when cuts are chosen poorly; (3) which structural properties of the query plan predict successful partitions; and (4) how fragment composition shapes the roles of the participating engines.

4.1 Partitioning wins more than expected

Figure 1 shows the CDF of the speedup achieved by the best partition over the best single-engine execution. We define a partition as beneficial when the speedup exceeds 1.05× — a conservative threshold that guards against measurement noise. Across all three workloads, **40.1% of queries** benefit from partitioning. The effect varies by workload: TPC-H shows the highest win rate at 55.5%, followed by JOB at 39.7% and SO at 26.5%. The magnitude of the gains can be substantial. Among the queries where partitioning wins, the median speedup is 1.46× on TPC-H, 1.10× on JOB, and 1.12× on SO. The tail gains are much larger: up to 9.66× on TPC-H, 14.87× on JOB, and 10.57× on SO. The P90 speedup on TPC-H reaches 5.47×, indicating that partitioning can deliver order-of-magnitude improvements for a nontrivial subset of queries. Overall, total workload times can be significantly reduced by partitioning, as shown in Table 1 (Ceiling gap).

Note that DuckDB is the faster single engine for 87% of queries, often by a wide margin (median 3–27× faster than DataFusion). Yet partitioning still wins for 40% of queries despite this engine asymmetry. The gains we observe arise from a narrow band of complementarity: per-operator latency measurements (Appendix A) show that DataFusion outperforms DuckDB primarily on SORT operators, while DuckDB is faster across all other operator types. Consistent with this, 95% of winning partitions across all three workloads place a SORT operator in the upper fragment F_2 assigned to DataFusion, confirming that this single axis of specialization is the dominant driver of partitioning gains in our engine pair.

[1] https://github.com/Soufiane-Barrada/dbtest26-supplement

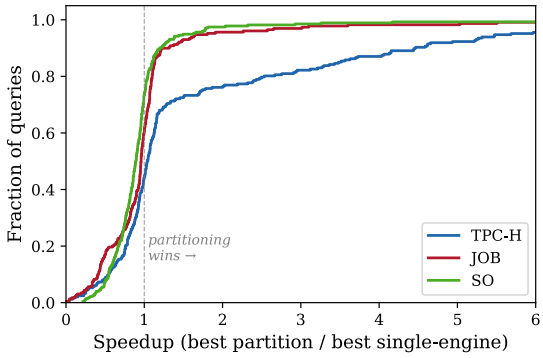


Figure 1: CDF of the speedup achieved by the best partition over the best single-engine execution. For 40% of queries, partitioning outperforms the best single engine.

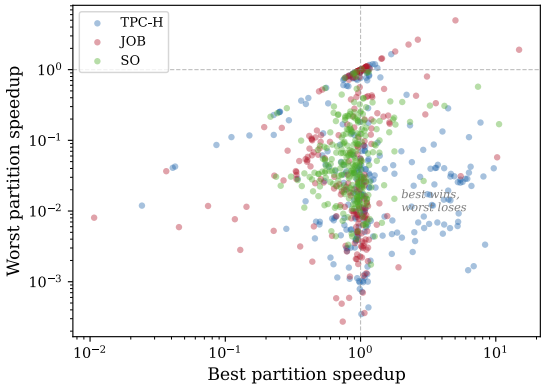


Figure 2: Best vs. worst partition speedup per query (log-log scale). Points right of the vertical dashed line benefit from partitioning at their best cut; points below the horizontal line suffer at their worst cut.

Table 2: Risk density: fraction of beneficial cuts per query.

Metric	TPC-H	JOB	SO
Beneficial cuts (% of all)	7.1%	5.2%	3.3%
P90 beneficial fraction / query	21.4%	41.1%	11.1%

In environments with more balanced engines, where complementary strengths span multiple operator types, the set of beneficial cut points should grow, and the ceiling gap should widen. Our 40% win rate and 25–33% workload reduction can therefore be read as a conservative estimate of the partitioning opportunity available to engine pairs with richer performance differentiation.

4.2 Risk of suboptimal partitioning

Worst-case risk. While the best partition per query can deliver order-of-magnitude speedups, the worst partition can be catastrophic. The median worst-cut penalty per query ranges from roughly 22× to 36× across datasets, and in the tail, some cuts are hundreds or even thousands of times slower than single-best engine. This is primarily driven by partitionings that produce massive intermediate results: among cuts that are 10× or more slower than single-engine execution, median intermediates range from 0.4 to 4.6 GB across workloads, compared to KB-scale transfers for winning partitions.

Figure 2 plots the best vs. worst partition speedup per query, illustrating this asymmetry. Both the gains and losses can be significant: the top-right quadrant (partitioning always helps regardless of cut choice) is nearly empty, while the bottom-right quadrant (best cut wins big, worst cut loses big) is densely populated. This stark risk/reward profile underscores the need for reliable partition selection—the upside is real and substantial, but the penalty for a poor cut is orders of magnitude worse. Any practical strategy for cross-engine partitioning must therefore rely on informed selection rather than naive enumeration or random choice.

Risk density. The partition space is not only asymmetric in magnitude but also sparse in opportunity. Across all three workloads, only 5.1% of all enumerated cuts are beneficial (7.1% on TPC-H, 5.2% on JOB, 3.3% on SO), as summarized in Table 2. At the 90th percentile the beneficial fraction reaches only 11–41%. Among the subset of queries where partitioning does help, the median share of winning cuts is just 10–12%, confirming that beneficial partitions are concentrated in a narrow structural band. This sparsity reinforces the risk profile: not only are poor cuts orders of magnitude worse than single-engine execution, but the probability of randomly selecting a good cut is below 10% even for queries known to benefit.

These two views are complementary: the extreme penalties quantify the *cost* of a poor choice, while the low density of beneficial cuts quantifies its *likelihood*—together they explain why informed selection is essential.

4.3 Anatomy of winning partitions

We identify four characteristics that strongly predict whether a partition will be beneficial.

1. Cut depth is the strongest predictor. Normalizing cut depth by plan height (0% = root, 100% = leaves), we find that cuts in the top 20% of the plan tree win 33–43% of the time depending on the workload, while cuts deeper than 40% almost never win (<5%). This indicates that the winning strategy is to place the cut near the root, so that the lower fragment (typically on DuckDB) handles the bulk of the computation and the upper fragment (typically on DataFusion) executes only a lightweight final stage.

2. Fragment balance confirms the pattern. Cuts where the lower fragment F_1 contains 80–100% of the plan nodes achieve win rates of 27–35%. Cuts giving F_1 fewer than 20% of the nodes win less than 2.5% of the time. The winning cuts are heavily asymmetric: DuckDB performs the heavy lifting, and DataFusion handles a small tail.

3. Intermediate result size gates feasibility. The total volume of the intermediate result at the cut point strongly influences outcomes. Cuts producing less than 10 KB win 15–22% of the time, while cuts producing over 1 GB win less than 3%. Among winning partitions, the median transfer is just a few KB on TPC-H and JOB, and approximately 4 MB on SO. A sweet spot exists for intermediate cardinality between 11 and 1,000 rows, where win rates peak at 19–31% on TPC-H and 11–20% on JOB.

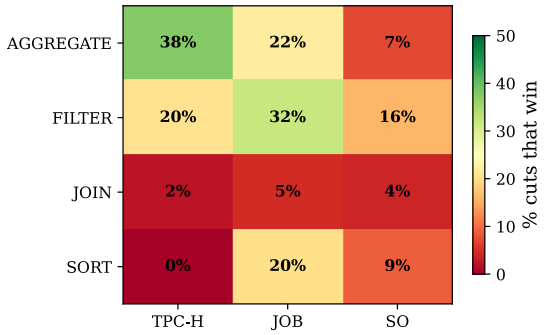


Figure 3: Win rate by operator type and workload.

4. Cut operator type matters, but varies by workload. Figure 3 shows the win rate for each cut operator type across workloads. On TPC-H, cuts at AGGREGATE nodes have the highest per-cut win rate (38%), and account for 72% of the best per-query partitions. On JOB, FILTER cuts dominate (32% win rate, 63% of winners). JOIN cuts rarely win across all workloads (<5%). This is consistent with the depth and balance findings: aggregates and filters near the plan root produce small intermediates, whereas join cuts tend to sit deeper and produce large intermediates.

Takeaways. Partitioning across heterogeneous engines is neither universally beneficial nor speculative. The benefit depends on predictable structural properties and the space of possible cuts contains severe pitfalls. The remaining challenge is to identify favorable partitions reliably without incurring the risks of poor cut selection.

4.4 Fragment composition and engine roles

Examining the operator composition of each fragment reveals *why* partitioning helps and surfaces two distinct mechanisms through which gains arise.

Operator-level specialization. When the upper fragment F_2 contains no JOIN operators, the win rate jumps to 17–25% (vs. 1–2% when F_2 contains joins). Similarly, the absence of aggregations or filters in F_2 correlates with higher win rates. The ideal F_2 is a tiny, single-operator fragment—a final projection, limit, or lightweight aggregation. Having a FILTER in the lower fragment F_1 is a strong positive signal: win rates reach 8–24% when F_1 contains a filter, vs. 2–3% without. This suggests that DuckDB's advantage lies in executing the full scan–join–filter–aggregate pipeline, consistent with the per-operator latency distributions in Appendix A. The partitioning gain arises from handing off the remaining operators to DataFusion once DuckDB has reduced the data. Almost all winning cuts assign DuckDB to F_1 and DataFusion to F_2 (100% on TPC-H and SO, 98% on JOB), confirming a consistent engine role specialization across all three workloads.

Optimizer complementarity. When a fragment is handed to an engine, it is re-optimized independently. Because the optimizer now sees a narrower schema, simpler join graph, and known intermediate cardinalities, the resulting plan can differ substantially from the subtree it would produce for the full query. For example, in query 28027 (found in `/queries/so_sqlstorm_test` in git), DuckDB runs the full query in 157 s and DataFusion in 221 s. Yet after a cut assigns a single base-table scan to DuckDB and hands the remaining 20 operators (including 6 joins) to DataFusion as F_2, DataFusion completes it in 15 s—just 6.7% of its own whole-query time despite containing 87% of the operators. Upon inspecting the physical plans of the fragments, we observe that DataFusion's optimizer appears to select better join orders on F_2 than for the full query. In git (`/resources/physical_plans_top10`), we include the physical plans for the whole-query and partitioned scenarios for 20 SQLStorm queries where partitioning brings benefit.

Table 3: Shallow vs. deep cuts among winning partitions. Deep cuts dominate SO savings via re-optimization.

		TPC-H	JOB	SO
Shallow (≤0.3)	% of best wins	91.7%	85.9%	59.6%
	Median speedup	2.00×	1.12×	1.14×
Deep (≥0.6)	% of best wins	2.5%	2.8%	21.2%
	Median speedup	3.12×	1.08×	1.66×
% of savings from deep		6.6%	0.1%	93.9%

Quantifying the two mechanisms. Table 3 separates these effects across workloads. Shallow cuts (relative depth ≤ 0.3) account for the majority of best-per-query wins on TPC-H (91.7%) and JOB (85.9%), consistent with the operator-specialization mechanism: DuckDB handles the heavy pipeline and DataFusion executes a lightweight tail. Deep cuts (relative depth ≥ 0.6), although they represent only 2.5–21% of best-per-query wins, contribute 93.9% of total time savings on SO—concentrated in a handful of long-running queries where re-optimization yields order-of-magnitude improvements. Deep cuts also exhibit higher median speedups on SO (1.66× vs. 1.14× for shallow cuts), indicating that the re-optimization effect, while less frequent, can be individually more impactful. These two mechanisms—shallow cuts for operator specialization, deep cuts for optimizer complementarity—are largely independent and jointly explain the breadth of the observed partitioning gains.

In summary, partitioning gains are not solely due to per-operator specialization but also arise from altered optimization at each engine level. These results serve as an empirical foundation for the benchmark protocol we propose in Section 6.

5 From Principle to Practice

A partition is beneficial when the compute gain from fragment specialization exceeds the accumulated boundary overhead: $\Delta_{\text{compute}} > T_{\text{transfer}} + T_{\text{materialization}}$. Two factors dominate this condition: *compute asymmetry* (engines exhibit complementary strengths across fragments) and *transfer dominance* (large intermediates make the inequality impossible to satisfy regardless of compute gains). Rather than evaluating this condition exactly—which would require perfect cross-engine cost estimation—we approximate it using structural filters derived from the analysis in Section 4.

1. Candidate filtering. Given a query plan with m edges and n engines, the exhaustive search space contains $O(m \cdot n^2)$ configurations. We identify two regions where winning partitions concentrate:

(1) **Shallow cuts.** Retain cuts in the top 30% of the plan tree (relative depth ≤ 0.3) where F_2 contains no JOIN operators.

Table 4: Two-filter strategy: try shallow and deep candidates. Geo. mean speedup is computed per-query over the best single-engine baseline.

Workload	Queries	Improved	Avg. cands	Time ↓	Geo. mean speedup
TPC-H	247	132 (53%)	2.1	24.9%	1.45×
JOB	229	84 (37%)	1.5	22.3%	1.10×
SO	275	86 (31%)	8.0	31.7%	1.06×

The threshold is relaxed from the 20% region where wins concentrate (Section 4.3) to conservatively capture borderline cases at minimal additional exploration cost.

(2) **Deep cuts.** Retain cuts at relative depth ≥ 0.6. These capture cases where the secondary engine re-plans the bulk of the query and finds a better execution strategy.

Both filters implicitly favor cuts with small intermediate results: shallow cuts sit near the plan root where most data reduction has already occurred, and deep cuts hand a single scan to F_1, producing intermediates bounded by base-table size.

These thresholds were chosen empirically and can be adjusted per system. Importantly, they target structural properties of the query plan—relative depth and intermediate result size—rather than workload-specific features. The filters also reflect different mechanisms by which the partitioning inequality is satisfied: shallow cuts succeed because T_{transfer} is negligible; deep cuts succeed because Δ_{compute} is large. These correspond directly to the two mechanisms identified in Section 4.4: operator specialization and optimizer complementarity, respectively. The fact that the same structural criteria identify winning partitions across three workloads with diverse schemas and query profiles suggests applicability beyond the workloads studied here, though systematic cross-workload validation of the specific thresholds remains an important direction for future work. On TPC-H and JOB, shallow cuts dominate. On SO, deep cuts drive the bulk of savings, accounting for 92% of the oracle total—concentrated in a handful of long-running queries (6 queries, 2.2% of the workload, contribute 89.5% of all SO savings). Table 4 summarizes the results of applying these filters on the datasets.

2. Reducing exploration cost with a query-level pre-filter. The filtering evaluates up to 8 candidates per query on SO, which is acceptable for recurring workloads but wasteful when most queries derive negligible benefit from partitioning. A lightweight pre-filter can reduce this cost by identifying queries unlikely to benefit—e.g., those with baseline latency below 1 s or low plan complexity—*before* enumerating any cuts.

3. Toward cost-based selection. The safe strategy above requires trial execution of all candidates, which may be acceptable for repeated workloads but is costly for ad-hoc queries. A cost-based variant would additionally estimate the intermediate result size at each candidate cut point and only attempt partitions where the estimated transfer is below a threshold. Our data shows that a threshold of 50 MB retains 54.5% of oracle-winning partitions, while 36.0% of the partitions attempted under this threshold actually improve over single-engine execution. However, the accuracy of this approach depends critically on cardinality estimation: while median estimated-to-actual transfer ratios are close to 1.0×, outlier estimation errors exceed 1,000×, indicating that cardinality estimation at cut points remains the key bottleneck for a fully cost-based partition selector. Promising directions for overcoming this bottleneck include ML-learned operator cost models that predict per-fragment latency across engines, dynamic programming over the plan tree guided by such models, and LLM-based partition selection that leverages query structure and natural-language plan descriptions to identify favorable cuts without explicit cost estimation.

Takeaways. Exhaustive enumeration establishes the attainable upper bound. The partitioning inequality defines the general decision condition. Two complementary structural filters—shallow cuts for fragment specialization, deep cuts for optimizer complementarity—approximate this condition in practice. Combined with a query-level pre-filter, this yields a low-cost approach that captures 86–96% of the oracle ceiling across all three workloads.

6 Implications for Testing and Benchmarking

The empirical findings in Sections 4 and 5 directly motivate each component of the protocol we propose below: the ceiling gap (Section 4.1) establishes what whole-query evaluation misses, the risk profile (Section 4.2) establishes what it hides, and the structural filters (Section 5) establish what practical exploration costs. Conventional benchmarking of heterogeneous systems assumes whole-query routing: a query is assigned to a single engine, and the reported metric is the best single-engine latency. Our results show that this practice systematically underestimates the performance potential—and obscures the risk profile—of multi-engine environments. We argue that heterogeneous benchmarking must become *partition-aware*, and propose a concrete evaluation protocol.

6.1 A partition-aware benchmark protocol

We propose that heterogeneous system evaluations report five metrics per workload, organized around three questions.

Q1: How much performance is left on the table? The *partition ceiling gap* measures the percentage reduction in total workload time achievable by an oracle that selects, for each query, the best of single-engine and best partition. A large gap signals unexploited compute asymmetry across engines and motivates partition-aware optimization. The *realized gap* measures the reduction achieved by a practical strategy (e.g., the structural filters from Section 5). Their ratio—the *capture ratio*—quantifies how much of the theoretical opportunity is practically accessible. Table 5 shows that the safe strategy captures 86-96% of the oracle.

Q2: What is the risk of partitioning? We propose reporting the *P95 worst-partition penalty*: the 95th-percentile ratio of the worst candidate's latency to the single-engine baseline. Under exhaustive search, the P95 penalty ranges from 140× (SO) to 846× (TPC-H). Structural filtering reduces this by 14–26×, bringing the P95 penalty down to 33–125× across workloads. The gap between filtered and unfiltered risk quantifies how much pruning contributes to safety.

Q3: How expensive is partition exploration? The *search efficiency* metric reports the average number of candidate partitions evaluated per query. The structural filter reduces the search space from ~10.6 candidates to ~1.8-8 per query. Together with the ceiling and risk

Table 5: Benchmark template. The ceiling gap quantifies untapped performance; the realized gap measures what a practical strategy achieves; the P95 penalty quantifies risk.

Metric	TPC-H	JOB	SO
Oracle ceiling gap (%)	26.8	25.8	33.0
Realized gap, filtered (%)	24.9	22.3	31.7
Capture ratio (%)	93.0	86.3	96.2
P95 penalty, exhaustive	846×	517×	140×
P95 penalty, filtered	33×	37×	125×
Avg. candidates / query	2.1	1.5	8

metrics, this lets benchmark users assess whether the performance opportunity justifies the exploration cost for their setting.

A partition-aware benchmark report would thus capture three dimensions that whole-query evaluation misses entirely—the performance ceiling, the risk of approaching it, and the cost of exploration—operationalized as the five concrete metrics in Table 5.

6.2 Reproducibility considerations

Applying this protocol to a new engine pair requires three capabilities: a common plan representation, a mechanism for cut enumeration, and a format for intermediate result transfer.

Plan representation. Our protocol operates on optimized logical operator trees rather than physical execution plans. Most analytical engines expose such plans via EXPLAIN variants or programmatic APIs (e.g., DataFusion's LogicalPlan API, Spark's queryExecution.optimizedPlan). Although naming conventions and operator granularity differ across systems, the core operator vocabulary—scan, filter, join, aggregate, sort, project—is largely shared. We normalize engine-specific operator labels into a common relational tree while preserving the coarsest operator granularity observed in either engine to avoid introducing artificial cut points when operators are fused in one system but not the other.

Cut enumeration. Given a normalized plan tree, cut enumeration is mechanical: each internal edge defines a candidate two-fragment partition. The key design choice is which engine's plan to use as the reference tree. We use a single canonical plan per query (DuckDB's logical plan) and re-plan each fragment independently on the assigned engine. This is an implementation choice; the protocol itself is agnostic to which engine provides the reference plan. An alternative is to enumerate cuts on each engine's plan separately and take the union, which may surface additional opportunities when optimizers produce structurally different trees for the same query.

Intermediate transfer. Partitioned execution requires materializing and transferring intermediate results between engines. Apache Arrow's columnar format [1] provides a practical lingua franca: both DuckDB and DataFusion support Arrow natively, and Arrow Flight or IPC can serve as the transfer mechanism. For engine pairs without native Arrow support, row-oriented serialization (e.g., CSV, Parquet) is an alternative at the cost of higher transfer overhead. The protocol should report the transfer format used, as it directly affects the measured ceiling gap. Most modern analytical engines already expose the required primitives: logical plans (via EXPLAIN or programmatic APIs), independent fragment execution, Arrow-based data interchange (native in DuckDB, DataFusion, and Spark; available via extensions in PostgreSQL), and per-fragment re-optimization. The protocol is therefore broadly applicable beyond the engine pair studied here.

Deployment considerations. The specific ceiling gaps, risk profiles, and search costs will differ across engine pairs—this is by design. Engine pairs with greater architectural diversity (e.g., a distributed engine paired with an embedded one) may exhibit larger ceiling gaps but also higher transfer costs, shifting the balance between the metrics. We recommend that adopters report the transfer format and deployment topology alongside the five metrics, as these directly affect the measured ceiling gap and risk profile.

7 Related Work

Cross-engine query processing has a long history in federated databases [5, 11] and has been revitalized by polystore systems. BigDAWG [10, 12] introduced engine "islands" for semantic completeness; PolyBase [8] implements split processing between SQL Server and Hadoop; CloudMdsQL [17] enables parallel polyglot queries across heterogeneous stores. On the optimization side, Deshpande and Hellerstein [7] proposed decoupled query optimization for federated databases, Simitsis et al. [24] addressed cost-based plan selection across multiple engines, and Haas et al. [14] studied optimization across diverse sources in Garlic. Wayang [3, 16] splits analytical workloads across engines using cost-based and ML-based strategies. These systems implement various partitioning heuristics but do not systematically evaluate the space of possible partitions to quantify the attainable benefit or risk.

A complementary line of work focuses on routing entire queries to a single backend. Frameworks such as Musketeer [13] target multiple engines but do not split individual queries, while learned query optimization approaches [15, 19, 20, 26, 27] predict engine performance from query features. These approaches address inter-query routing but leave the intra-query partitioning ceiling unmeasured.

On the benchmarking side, the "one size fits all" critique [25] motivated specialized engines and, consequently, the need for cross-engine evaluation. Subsequent work has emphasized rigorous and reproducible benchmarking [6, 9, 21], and [4] identifies cross-engine optimization as an open challenge. However, existing benchmarks assume whole-query routing. Our work complements all three lines by providing an empirical characterization of the partitioning ceiling and risk surface, and proposing a benchmark protocol that captures dimensions current evaluations miss.

8 Conclusions

Whole-query evaluation systematically underestimates heterogeneous multi-engine systems: it underestimates their performance potential and obscures their risk profile in equal measure. Our exhaustive study of two-fragment partitioning confirms this: partitioning outperforms optimal single-engine routing for 40% of queries (up to 14.87×), while poor cuts incur catastrophic penalties. A structurally guided strategy captures 86–96% of the oracle ceiling, and the partition-aware benchmark protocol we propose surfaces the performance ceiling, risk, and exploration cost that current evaluations miss.

References

[1] Apache Software Foundation. 2026. Apache Arrow: A Cross-Language Development Platform for In-Memory Analytics. https://arrow.apache.org/. Open-source project.

[2] Apache Software Foundation. 2026. Apache DataFusion. https://arrow.apache.org/datafusion/. Accessed: 2026-02-28.

[3] Kaustubh Beedkar, Bertty Contreras-Rojas, Haralampos Gavriilidis, Zoi Kaoudi, Volker Markl, Rodrigo Pardo-Meza, and Jorge-Arnulfo Quiané-Ruiz. 2023. Apache Wayang: A Unified Data Analytics Framework. *SIGMOD Rec.* 52, 3 (Nov. 2023), 30–35. doi:10.1145/3631504.3631510

[4] Carlyna Bondiombouy and Patrick Valduriez. 2016. Query Processing in Multistore Systems: An Overview. *International Journal of Cloud Computing* 5, 4 (2016), 309–346.

[5] Sudarshan S. Chawathe, Hector Garcia-Molina, Joachim Hammer, Kelly Ireland, Yannis Papakonstantinou, Jeffrey D. Ullman, and Jennifer Widom. 1994. The TSIMMIS Project: Integration of Heterogeneous Information Sources. In *Proceedings of the 10th Meeting of the Information Processing Society of Japan, Tokyo, Japan, October 1994*. 7–18.

[6] Carlo Curino, Evan Jones, Yang Zhang, and Sam Madden. 2010. Schism: a workload-driven approach to database replication and partitioning. *Proc. VLDB Endow.* 3, 1–2 (Sept. 2010), 48–57. doi:10.14778/1920841.1920853

[7] Amol Deshpande and Joseph M. Hellerstein. 2002. Decoupled Query Optimization for Federated Database Systems . In *Proceedings 18th International Conference on Data Engineering*. IEEE Computer Society, Los Alamitos, CA, USA, 0716. doi:10.1109/ICDE.2002.994788

[8] David J. DeWitt, Alan Halverson, Rimma Nehme, Srinath Shankar, Josep Aguilar-Saborit, Artin Avanes, Miro Flasza, and Jim Gramling. 2013. Split query processing in polybase. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data* (New York, New York, USA) *(SIGMOD '13)*. Association for Computing Machinery, New York, NY, USA, 1255–1266. doi:10.1145/2463676.2463709

[9] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudre-Mauroux. 2013. OLTP-Bench: an extensible testbed for benchmarking relational databases. *Proc. VLDB Endow.* 7, 4 (Dec. 2013), 277–288. doi:10.14778/2732240.2732246

[10] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, Magdalena Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. 2015. The BigDAWG Polystore System. *ACM SIGMOD Record* 44, 2 (2015), 11–16.

[11] R. Fagin, P.M. Schwarz, L.M. Haas, W.F. Cody, J. Thomas, D. Petkovic, M.J. Carey, A.W. Luniewski, W. Niblack, M. Flickner, M. Arya, E.L. Wimmers, and J.H. Williams. 1995. Towards heterogeneous multimedia information systems: the Garlic approach . In *Research Issues in Data Engineering, International Workshop on*. IEEE Computer Society, Los Alamitos, CA, USA, 124. doi:10.1109/RIDE.1995.378736

[12] Vijay Gadepally, Peter Chen, Jennie Duggan, Aaron J. Elmore, Brandon Haynes, Jeremy Kepner, Samuel Madden, Timothy Mattson, and Michael Stonebraker. 2016. The BigDAWG Polystore System and Architecture. In *IEEE High Performance Extreme Computing Conference (HPEC)*. 1–6.

[13] Ionel Gog, Malte Schwarzkopf, Natacha Crooks, Matthew P. Grosvenor, Allen Clement, and Steven Hand. 2015. Musketeer: all for one, one for all in data processing systems. In *Proceedings of the Tenth European Conference on Computer Systems* (Bordeaux, France) *(EuroSys '15)*. Association for Computing Machinery, New York, NY, USA, Article 2, 16 pages. doi:10.1145/2741948.2741968

[14] Laura M. Haas, Donald Kossmann, Edward L. Wimmers, and Jun Yang. 1997. Optimizing Queries Across Diverse Data Sources. In *Proceedings of the 23rd International Conference on Very Large Data Bases (VLDB '97)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 276–285.

[15] Benjamin Hilprecht and Carsten Binnig. 2022. Zero-shot cost models for out-of-the-box learned cost prediction. *Proc. VLDB Endow.* 15, 11 (July 2022), 2361–2374. doi:10.14778/3551793.3551799

[16] Zoi Kaoudi, Jorge-Arnulfo Quiane-Ruiz, Bertty Contreras-Rojas, Rodrigo Pardo-Meza, Anis Troudi, and Sanjay Chawla. 2020. ML-based Cross-Platform Query Optimization . In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE Computer Society, Los Alamitos, CA, USA, 1489–1500. doi:10.1109/ICDE48307.2020.00132

[17] Pavlos Kranas, Boyan Kolev, Oleksandra Levchenko, Esther Pacitti, Patrick Valduriez, Ricardo Jiménez-Peris, and Marta Patiño Martinez. 2021. Parallel query processing in a polystore. *Distrib. Parallel Databases* 39, 4 (Dec. 2021), 939–977. doi:10.1007/s10619-021-07322-5

[18] Sebastian Kruse, Zoi Kaoudi, Bertty Contreras-Rojas, Sanjay Chawla, Felix Naumann, and Jorge-Arnulfo Quiané-Ruiz. 2020. RHEEMix in the data jungle: a cost-based optimizer for cross-platform systems. *The VLDB Journal* 29, 6 (May 2020), 1287–1310. doi:10.1007/s00778-020-00612-x

[19] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 1275–1288. doi:10.1145/3448016.3452838

[20] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: a learned query optimizer. *Proc. VLDB Endow.* 12, 11 (July 2019), 1705–1718. doi:10.14778/3342263.3342644

[21] Andrew Pavlo. 2017. The Benchmarking of Data Systems. *Commun. ACM* 60, 8 (2017), 16–19.

[22] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1981–1984. doi:10.1145/3299869.3320212

[23] Tobias Schmidt, Viktor Leis, Peter Boncz, and Thomas Neumann. 2025. SQLStorm: Taking Database Benchmarking into the LLM Era. *Proc. VLDB Endow.* 18, 11 (July 2025), 4144–4157. doi:10.14778/3749646.3749683

[24] Alkis Simitsis, Kevin Wilkinson, Malu Castellanos, and Umeshwar Dayal. 2012. Optimizing analytic data flows for multiple execution engines. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data* (Scottsdale, Arizona, USA) *(SIGMOD '12)*. Association for Computing Machinery, New York, NY, USA, 829–840. doi:10.1145/2213836.2213963

[25] Michael Stonebraker and Ugur Çetintemel. 2018. *"One size fits all": an idea whose time has come and gone.* Association for Computing Machinery and Morgan & Claypool, 441–462. https://doi.org/10.1145/3226595.3226636

[26] András Strausz, Niels Pardon, and Ioana Giurgiu. 2025. A Learned Cost Model-based Cross-engine Optimizer for SQL Workloads. *arXiv preprint arXiv:2506.02802* (2025). arXiv:2506.02802 [cs.DB] https://arxiv.org/abs/2506.02802

[27] Rong Zhu, Wei Chen, Bolin Ding, Xingguang Chen, Andreas Pfadler, Ziniu Wu, and Jingren Zhou. 2023. Lero: A Learning-to-Rank Query Optimizer. *Proc. VLDB Endow.* 16, 6 (Feb. 2023), 1466–1479. doi:10.14778/3583140.3583160

A Operator-Level Latency Distributions

To better understand the source of compute asymmetry between DuckDB and DataFusion, we analyze the execution-time distributions of individual operator types across the workload. For each operator class (Aggregate, Filter, Join, Sort), we collect per-operator runtimes from whole-query executions and plot their empirical distributions on a logarithmic time scale.

In all figures, the x-axis shows execution time (log scale), and the y-axis reports the percentage of operator instances per latency bin.

A.1 Aggregate operators

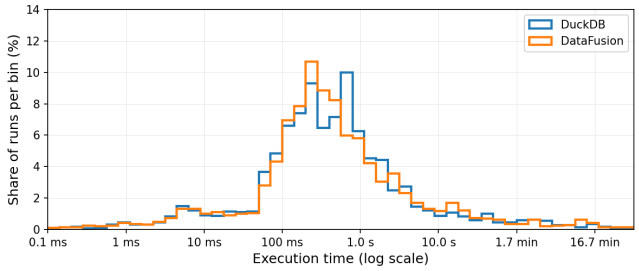


Figure 4: Execution-time distribution of Aggregate operators (log scale).

DuckDB dominates across most latency bins for aggregate operators. DataFusion shows slightly higher density in certain mid-range bins, but the majority of aggregate executions are consistently faster on DuckDB. This aligns with the shallow-cut results in Section 4: when aggregates appear near the plan root, DuckDB typically executes them more efficiently.

A.2 Filter operators

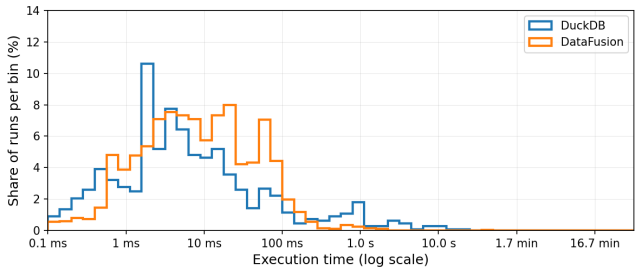


Figure 5: Execution-time distribution of Filter operators (log scale).

Filter operators exhibit a similar pattern: DuckDB's distribution is shifted left relative to DataFusion across most bins, indicating generally lower latency. Since filters often reduce intermediate cardinality, this asymmetry helps explain why shallow cuts at filter boundaries frequently succeed—DuckDB performs early data reduction efficiently before handing off a smaller fragment.

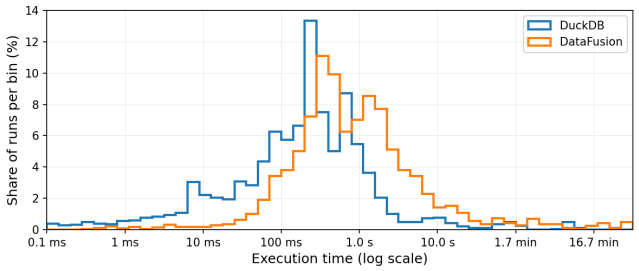


Figure 6: Execution-time distribution of Join operators (log scale).

A.3 Join operators

Join operators show broader distributions for both engines, with DuckDB generally outperforming DataFusion in the central mass of executions. Variance is higher here, and long-tail behavior is visible. This variability contributes to the effects discussed in Section 4.4: fragment-level re-optimization can materially change join order and execution strategy, occasionally yielding substantial gains for deep cuts.

A.4 Sort operators

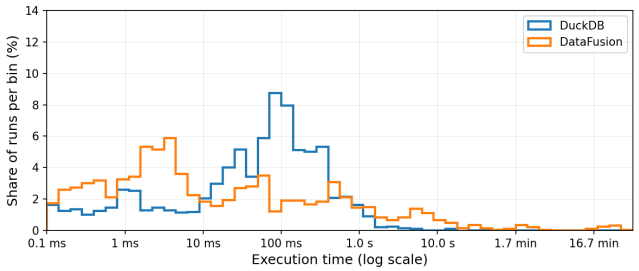


Figure 7: Execution-time distribution of Sort operators (log scale).

Sort is the primary operator type where DataFusion demonstrates competitive or superior performance relative to DuckDB. Although this specialization is narrow, it is sufficient to generate measurable partitioning gains even when DuckDB dominates most other operator classes. As argued in Section 4, this limited complementarity implies that the observed 25–33% ceiling gap likely represents a conservative lower bound: broader operator specialization would expand the feasible partitioning region.